

Choix techniques — TP Infrastructure Docker Avancée

Vue d'ensemble

Ce document détaille les choix techniques allant **au-delà du scope minimal du TP**, notamment en matière de sécurité, fiabilité et bonnes pratiques DevOps.

Sommaire

1. [Sécurité des conteneurs](#)
2. [Segmentation réseau \(VMAp\)](#)
3. [TLS — Choix cryptographiques](#)
4. [Headers de sécurité HTTP](#)
5. [Rate Limiting](#)
6. [Authentification — Choix supplémentaires](#)
7. [Health checks et dépendances](#)
8. [Rotation des logs conteneurs](#)
9. [Volumes et montages](#)
10. [Indexation Elasticsearch](#)
11. [Rétention des logs \(ILM\)](#)
12. [Orchestration du déploiement \(VMAp\)](#)
13. [Alerting Telegram](#)
14. [Récapitulatif OWASP](#)
15. [Annexe — Captures d'écran](#)

1. Sécurité des conteneurs

1.1 Principe du moindre privilège

Chaque conteneur applique une politique de restriction maximale des privilèges :

```
security_opt:  
  - no-new-privileges:true  
cap_drop:  
  - ALL
```

- **no-new-privileges** : empêche tout processus fils d'acquérir plus de privilèges que son parent (même via `setuid`). Correspond au **CIS Docker Benchmark Level 2**.
- **cap_drop: ALL** : supprime toutes les capacités Linux du conteneur. Seules les capacités strictement nécessaires sont réajoutées au cas par cas :
 - Uptime Kuma : **SETGID** + **SETUID** (changement d'utilisateur interne)
 - Filebeat : **DAC_READ_SEARCH** (lecture des fichiers de logs des autres conteneurs)

Pourquoi ? Un conteneur compromis avec toutes les capacités pourrait manipuler le réseau, monter des systèmes de fichiers, ou escalader vers le host. En supprimant tout, on réduit drastiquement la surface

d'attaque.

1.2 Limites de ressources

Chaque conteneur a des limites mémoire et CPU :

Service	Mémoire	CPU	Justification
Traefik	256m	0.5	Reverse proxy léger
MariaDB-WP	512m	1.0	Opérations I/O intensives
MariaDB-KC	512m	1.0	Opérations I/O intensives
WordPress	512m	1.0	PHP + Apache
phpMyAdmin	256m	0.5	Interface légère
Keycloak	768m	1.0	JVM gourmande en mémoire
Filebeat	256m	0.5	Agent léger
Elasticsearch	1g	1.5	Indexation + recherche
Logstash	512m	1.0	Parsing des logs
Kibana	1g	1.0	Interface web + requêtes ES
Uptime Kuma	256m	0.5	Monitoring léger

Pourquoi ? Sans limites, un seul conteneur peut consommer toutes les ressources du host et provoquer un déni de service sur les autres services (fork bomb, fuite mémoire, etc.).

1.3 Versions d'images épinglées

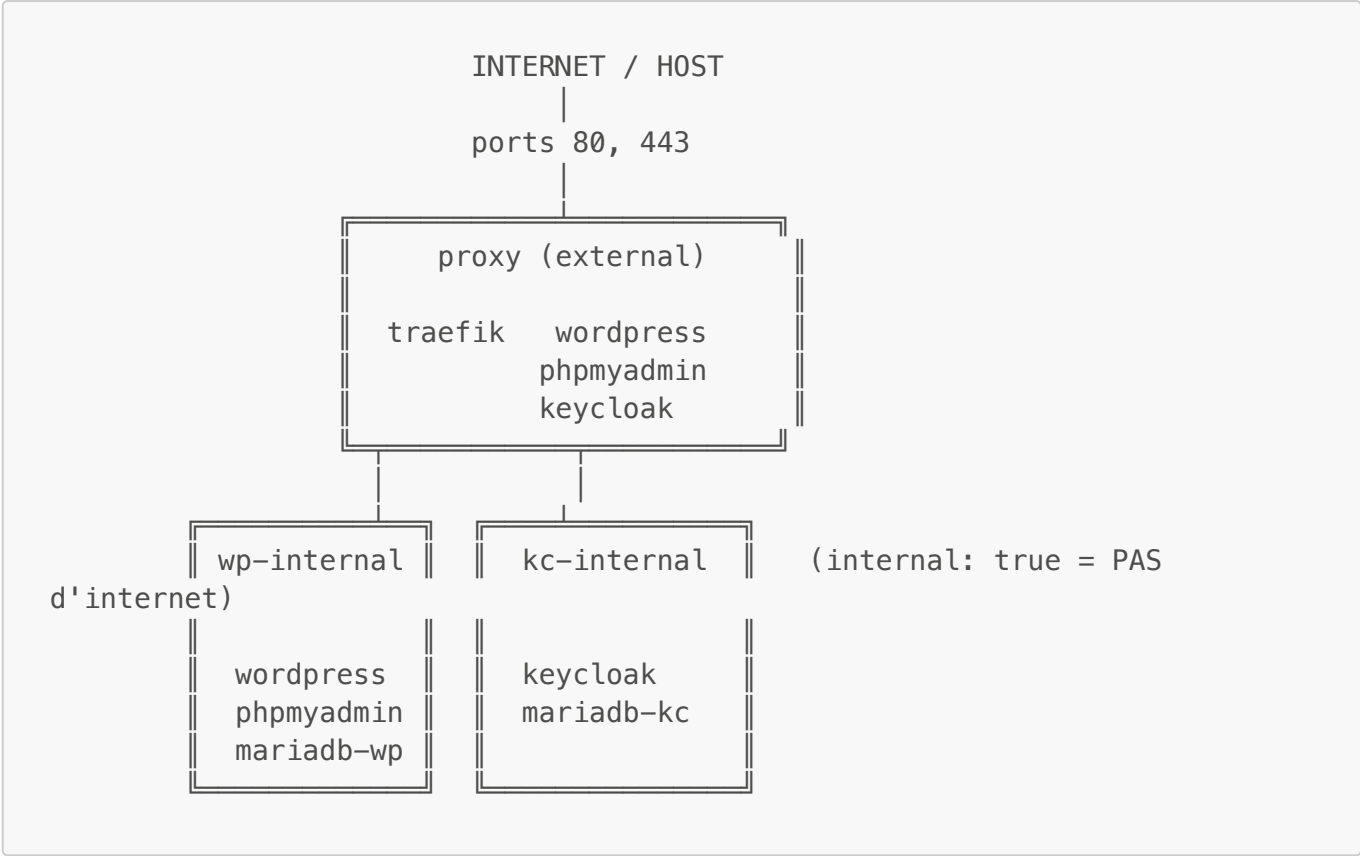
Aucune image n'utilise le tag `latest`. Toutes sont épinglées :

```
traefik:v3.6
mariadb:11
wordpress:6-apache
keycloak:26.0
filebeat:8.17.0
elasticsearch:8.17.0
```

Pourquoi ? Le tag `latest` est mutable — un `docker pull` peut ramener une version avec des breaking changes ou des vulnérabilités. L'épinglage garantit des builds **reproductibles** et un contrôle sur les mises à jour.

2. Segmentation réseau (VMApp)

VMApp est découpé en **4 stacks Docker Compose indépendants** (traefik, wordpress, keycloak, filebeat) avec **3 réseaux isolés** :



Matrice d'accès réseau :

Service	proxy	wp-internal	kc-internal	Internet
traefik	oui	—	—	oui (ports 80/443)
wordpress	oui	oui	—	oui (via proxy)
phpmyadmin	oui	oui	—	oui (via proxy)
mariadb-wp	—	oui	—	non
keycloak	oui	—	oui	oui (via proxy)
mariadb-kc	—	—	oui	non
filebeat	défaut	—	—	réseau par défaut

Propriétés de sécurité :

- **proxy** : réseau Docker **externe** partagé entre les 4 stacks, utilisé par Traefik pour le routage (`--providers.docker.network=proxy`)
- **wp-internal** (`internal: true`) : réseau fermé sans accès internet. `mariadb-wp` y est **exclusivement** connecté — aucune route vers l'extérieur ni vers le stack Keycloak
- **kc-internal** (`internal: true`) : idem pour `mariadb-kc` — aucune route vers le stack WordPress ni vers internet
- **Chaque base de données a son propre utilisateur dédié** : `wpdbadmin` pour WordPress, `kcdbadmin` pour Keycloak (plus de partage de credentials)

Pourquoi ce découpage ?

- **Isolation inter-stacks** : une compromission de WordPress ne donne **aucun accès** à la base Keycloak (et inversement)
- **Databases sans internet** : `internal: true` garantit que les bases de données ne peuvent pas exfiltrer de données ni être contactées depuis l'extérieur
- **Blast radius réduit** : chaque stack peut être arrêté/redémarré indépendamment sans impacter les autres

Aucun port exposé inutilement

MariaDB, Elasticsearch, Kibana, WordPress n'exposent **aucun port au host**. Tout le trafic passe par Traefik qui agit comme unique point d'entrée.

3. TLS — Choix cryptographiques

```
Root CA (4096-bit RSA, SHA256)
├── Server Certificate (2048-bit RSA, SHA256)
│   ├── *.tp.local (wildcard)
│   ├── localhost
│   ├── host.lima.internal
│   └── IPs: 127.0.0.1, 192.168.5.2, 192.168.5.15
```

- **4096-bit pour la CA** : clé plus forte car elle signe tous les certificats
- **2048-bit pour le serveur** : bon compromis performance/sécurité
- **SHA256** : algorithme de hachage standard (SHA1 est déprécié)
- **SAN (Subject Alternative Names)** : format moderne, le champ CN seul est déprécié
- `extendedKeyUsage = serverAuth` : le certificat ne peut servir qu'à l'authentification serveur
- `basicConstraints = CA:FALSE` : empêche l'utilisation du certificat serveur comme CA

4. Headers de sécurité HTTP

Traefik injecte des headers de sécurité sur **toutes les réponses** via un middleware :

```
security-headers:
  headers:
    browserXssFilter: true # X-XSS-
Protection: 1; mode=block
    contentTypeNosniff: true # X-Content-Type-
Options: nosniff
    frameDeny: true # X-Frame-Options:
DENY
    stsSeconds: 31536000 # HSTS 1 an
    stsIncludeSubdomains: true # HSTS sous-
domaines
    customFrameOptionsValue: "SAMEORIGIN" # Frame same-
origin
    referrerPolicy: "strict-origin-when-cross-origin" # Referrer limité
```

Header	Protection
X-XSS-Protection	Bloque les attaques XSS réfléchies (legacy)
X-Content-Type-Options: nosniff	Empêche le MIME-type sniffing (ex: exécuter un JS déguisé en image)
X-Frame-Options: DENY	Empêche le clickjacking via iframe
Strict-Transport-Security	Force HTTPS pendant 1 an (protection contre sslstrip)
Referrer-Policy	Limite les informations envoyées dans le header Referer

Cas particulier Keycloak : un middleware dédié (`security-headers-keycloak`) utilise `SAMEORIGIN` au lieu de `DENY` pour `X-Frame-Options`, car les formulaires de login Keycloak peuvent nécessiter d'être intégrés en iframe.

5. Rate Limiting

```
rate-limit:
  rateLimit:
    average: 100
    burst: 200
```

- **100 requêtes/seconde** en moyenne par source, **200** en pic (burst)
- Appliqué sur **tous les routers** Traefik

Pourquoi ? Protège contre les attaques DDoS applicatif, le brute-force sur les pages de login, et le scraping agressif.

6. Authentification — Choix supplémentaires

Bcrypt pour BasicAuth phpMyAdmin

```
# Format: user:$$2y$$05$$[bcrypt_hash]
```

Bcrypt (coût 05) plutôt que MD5/SHA1 : résistant aux attaques par dictionnaire. Double couche d'authentification (BasicAuth Traefik + login MariaDB).

Elasticsearch — xpack.security

```
xpack.security.enabled: true
xpack.security.enrollment.enabled: false
```

- Authentification obligatoire sur l'API Elasticsearch (non requis par défaut)
- Utilisateurs séparés : `elastic` (admin) et `kibana_system` (accès Kibana)

Mots de passe forts

Tous les mots de passe font **32+ octets** encodés en base64, générés aléatoirement.

7. Health checks et dépendances

Health checks adaptés par service

Service	Méthode	Particularité
Traefik	<code>wget</code> sur port dédié 8082	Port de ping séparé du trafic
MariaDB	<code>healthcheck.sh --innodb_initialized</code>	Vérifie que InnoDB est prêt
WordPress	<code>curl -f http://localhost:80/</code>	Vérifie le serveur HTTP
Keycloak	TCP socket sur <code>/health/ready</code>	Endpoint de health dédié
Elasticsearch	<code>curl</code> sur <code>/_cluster/health</code> avec auth	Vérifie le cluster
Kibana	<code>curl</code> sur <code>/api/status</code> avec auth	Vérifie l'API
Uptime Kuma	<code>curl</code> sur port 3001	Vérifie l'interface web

Chaîne de dépendances avec `service_healthy`

```
depends_on:
  mariadb-wp:
    condition: service_healthy
```

Les services ne démarrent que lorsque leurs dépendances sont **healthy** (pas juste "started") :

Intra-stack (via `depends_on`) :

```
mariadb-wp (healthy) → WordPress, phpMyAdmin
mariadb-kc (healthy) → Keycloak
Elasticsearch (healthy) → Logstash, Kibana
```

Inter-stacks (via `setup.sh`, orchestration séquentielle) :

```
Réseau proxy → Traefik (healthy) → WordPress stack + Keycloak stack →
Filebeat
```

Pourquoi `service_healthy` plutôt que `service_started` ? Un conteneur "started" ne signifie pas que le service est prêt. WordPress qui démarre avant que MariaDB soit prêt crashe avec une erreur de connexion.

8. Rotation des logs conteneurs

```
logging:
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"
```

- **30 Mo maximum** par conteneur (3 fichiers × 10 Mo)
 - Empêche le remplissage du disque en cas de logs excessifs
-

9. Volumes et montages

Volumes nommés par stack

Chaque stack gère ses propres volumes, préfixés automatiquement par Docker Compose :

```
# Stack WordPress
volumes:
  db-data:      # Données MariaDB-WP
  wp-data:      # Fichiers WordPress

# Stack Keycloak
volumes:
  kc-db-data:   # Données MariaDB-KC
  kc-data:      # Configuration Keycloak
```

Pourquoi des volumes nommés plutôt que des bind mounts ? Gérés par Docker (portables, backupables), pas de dépendance au chemin du host, survivent à `docker compose down`.

Montages en lecture seule

Tous les fichiers de configuration sont montés en `:ro` (read-only) :

```
- ./traefik/dynamic.yml:/etc/traefik/dynamic.yml:ro
- ./certs:/certs:ro
- ./filebeat/filebeat.yml:ro
```

Pourquoi ? Un conteneur compromis ne peut pas modifier sa propre configuration ou les certificats TLS.

10. Indexation Elasticsearch

```
index => "%{[@metadata][index_prefix]}-%{+YYYY.MM.dd}"
```

Indices journaliers par service : `traefik-access-2026.02.27`, `wordpress-2026.02.27`, `mariadb-wp-2026.02.27`, etc.

Pourquoi ? Permet de supprimer facilement les vieux logs (ex: `DELETE traefik-access-2026.01.*`), d'appliquer des politiques de rétention, et d'optimiser les recherches sur une plage de dates.

11. Rétention des logs (ILM)

Elasticsearch Index Lifecycle Management (ILM) gère automatiquement la suppression des vieux indices pour éviter de remplir le disque :

Policy	Rétention	Indices concernés	Justification
<code>logs-app-30d</code>	30 jours	<code>traefik-access-*</code> , <code>wordpress-*</code> , <code>keycloak-*</code>	Logs applicatifs utiles pour debug et audit
<code>logs-db-14d</code>	14 jours	<code>mariadb-*</code> , <code>mariadb-wp-*</code> , <code>mariadb-kc-*</code>	Très volumineux (~75k docs/jour), peu consultés au-delà de 2 semaines
<code>logs-misc-7d</code>	7 jours	<code>misc-*</code>	Logs non catégorisés, faible valeur

Chaque policy est liée à un **index template** qui l'applique automatiquement à tout nouvel index correspondant au pattern. Les indices existants sont également rattachés.

Les templates fixent aussi `number_of_replicas: 0` (approprié pour un cluster single-node — évite le statut `yellow` permanent).

12. Orchestration du déploiement (VMApp)

Le `setup.sh` de VMApp orchestre le démarrage séquentiel avec **attente de healthchecks** entre chaque étape :

```
Réseau proxy → Traefik (wait healthy) → WordPress + Keycloak (wait healthy) → Filebeat
```

Scripts complémentaires :

- `teardown.sh` : arrêt propre dans l'ordre inverse
- `migrate-db.sh` : migration one-shot depuis l'ancien stack monolithique (dump → import avec FK checks désactivés)

13. Alerting Telegram

Uptime Kuma envoie des notifications Telegram en temps réel :

- **DOWN** : alerte immédiate quand un service tombe
- **UP** : confirmation quand le service revient
- Lié à **tous les monitors** (8 services surveillés)

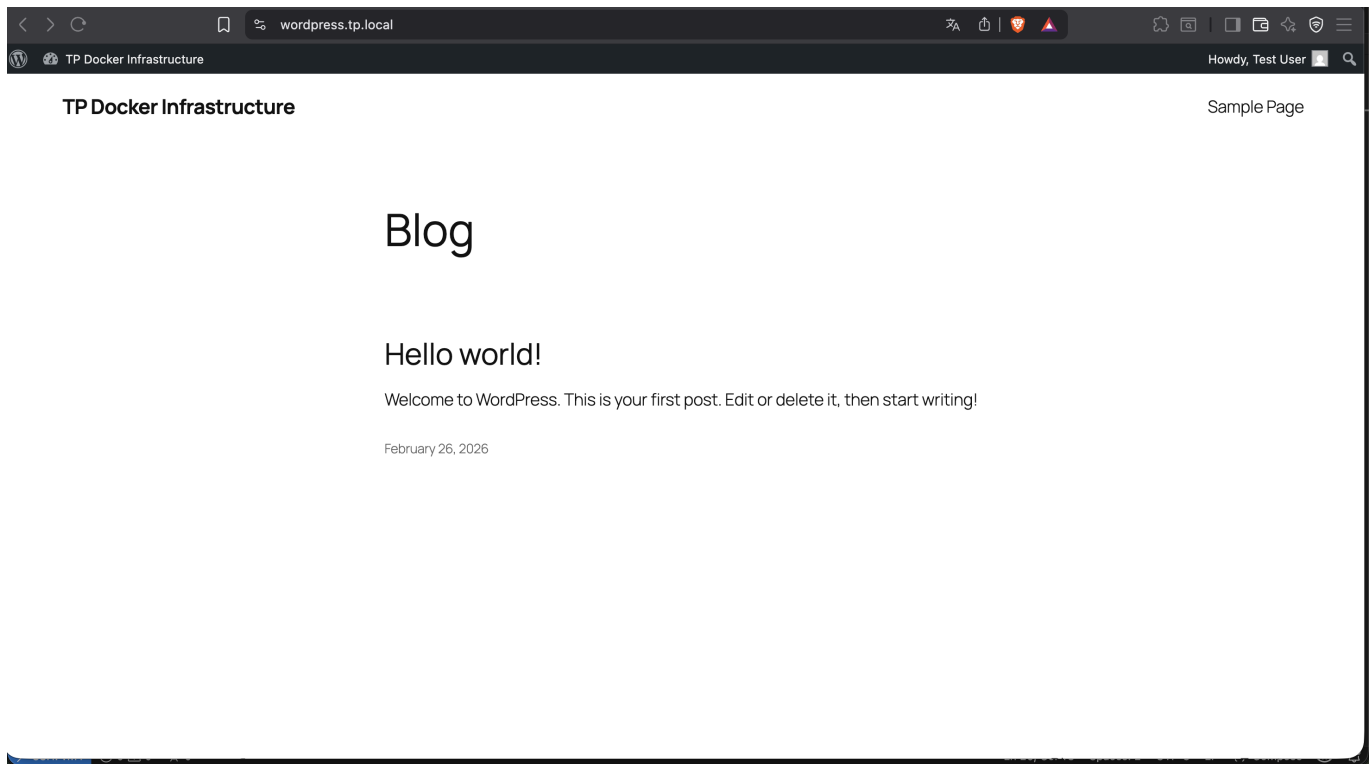
Pourquoi Telegram plutôt qu'email ? Notification instantanée sur mobile, pas de dépendance à un serveur SMTP externe, configuration plus simple (token + chat ID vs credentials SMTP + DNS).

Récapitulatif des protections OWASP

OWASP Top 10	Protection mise en place
A01 — Broken Access Control	Isolation réseau par stack (<code>internal: true</code>), credentials DB séparés, auth Elasticsearch
A02 — Cryptographic Failures	HSTS 1 an, certificats SHA256/4096-bit
A03 — Injection	Parsing structuré des logs (Logstash)
A05 — Security Misconfiguration	<code>no-new-privileges</code> , <code>cap_drop: ALL</code> , pas de ports inutiles
A06 — Vulnerable Components	Images épinglées, pas de <code>latest</code>
A07 — Auth Failures	Bcrypt, rate limiting
A09 — Logging & Monitoring	Indexation journalière + alertes Telegram

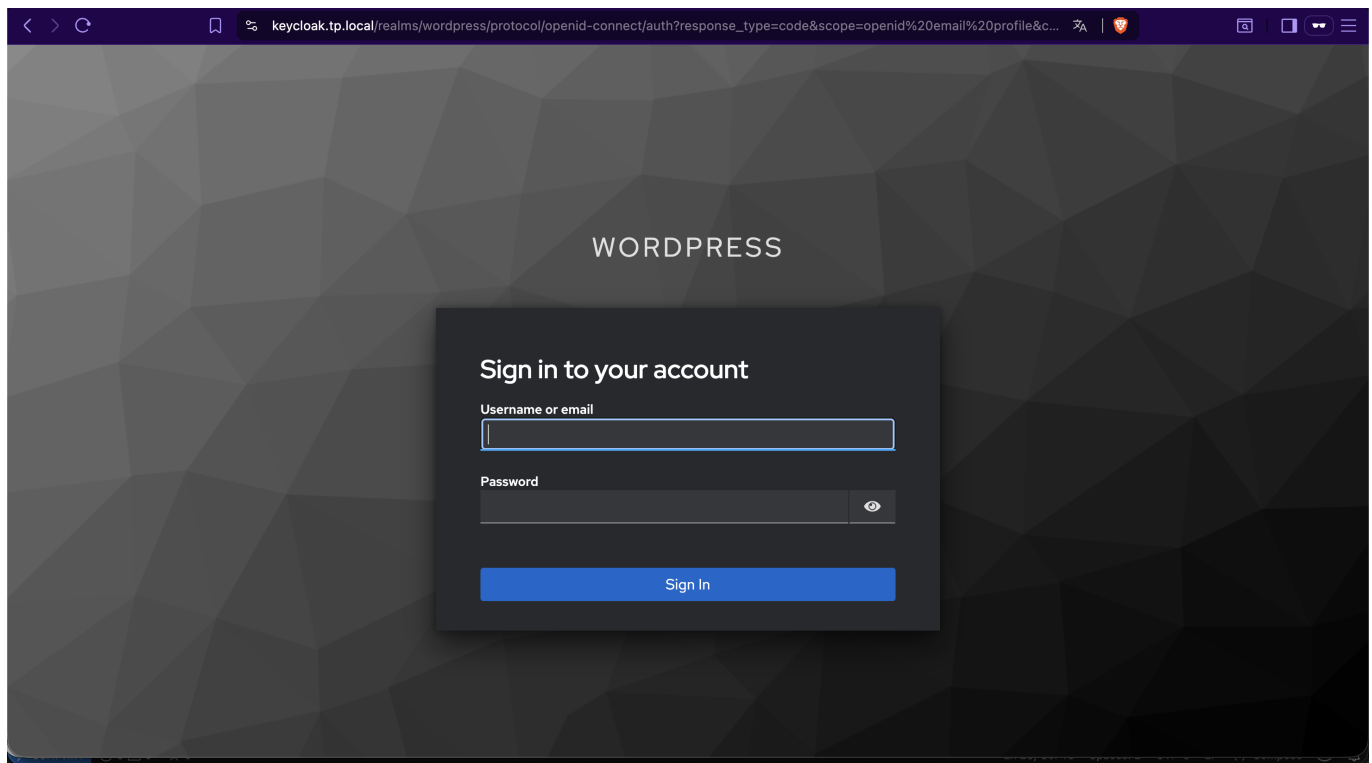
Annexe — Captures d'écran

VM-App : WordPress

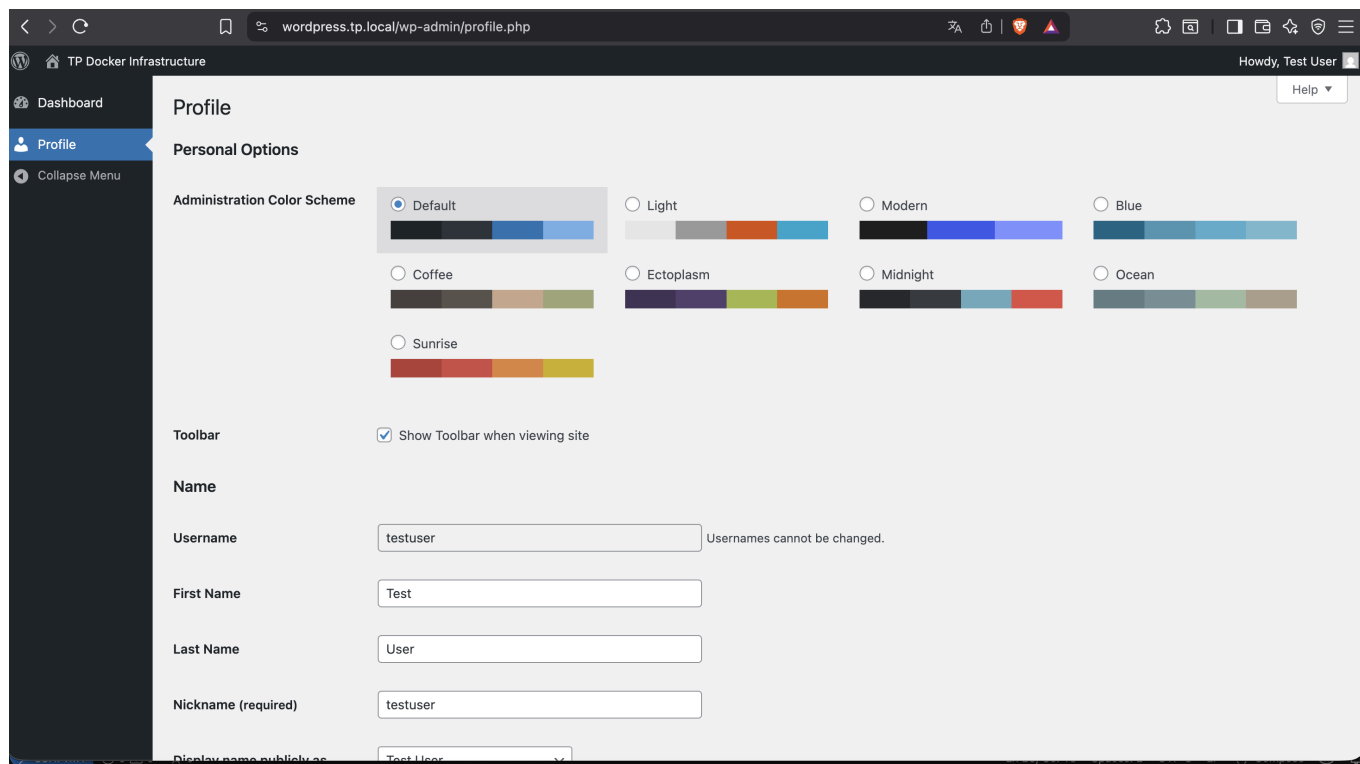


Page d'accueil du site WordPress servi sur **wordpress.tp.local**. L'utilisateur "Test User" est connecté via Keycloak (barre admin visible en haut). Le post "Hello world!" confirme le bon fonctionnement du CMS.

VM-App : Authentification Keycloak (OpenID Connect)

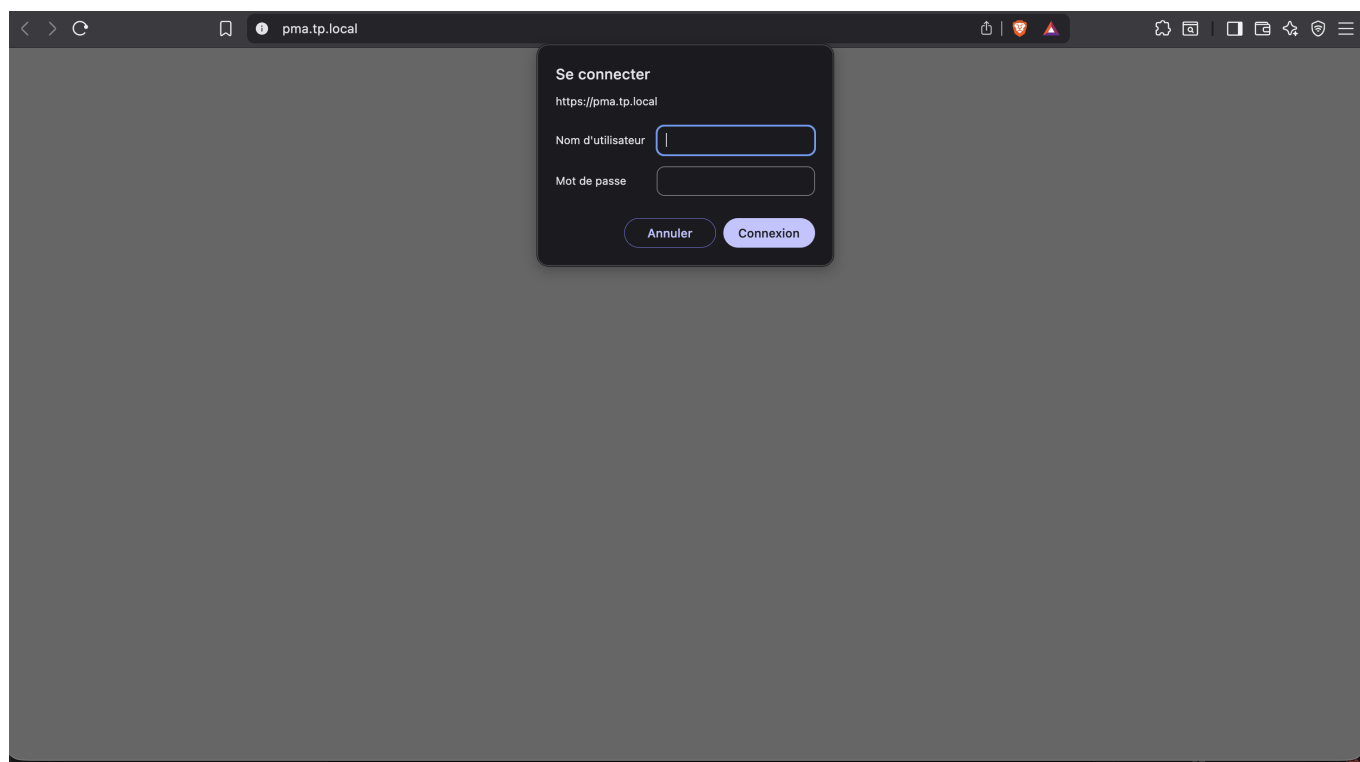


Page de connexion du realm "WORDPRESS" sur **keycloak.tp.local**. C'est cette page qui s'affiche quand un utilisateur clique sur "Login with OpenID Connect" depuis WordPress.

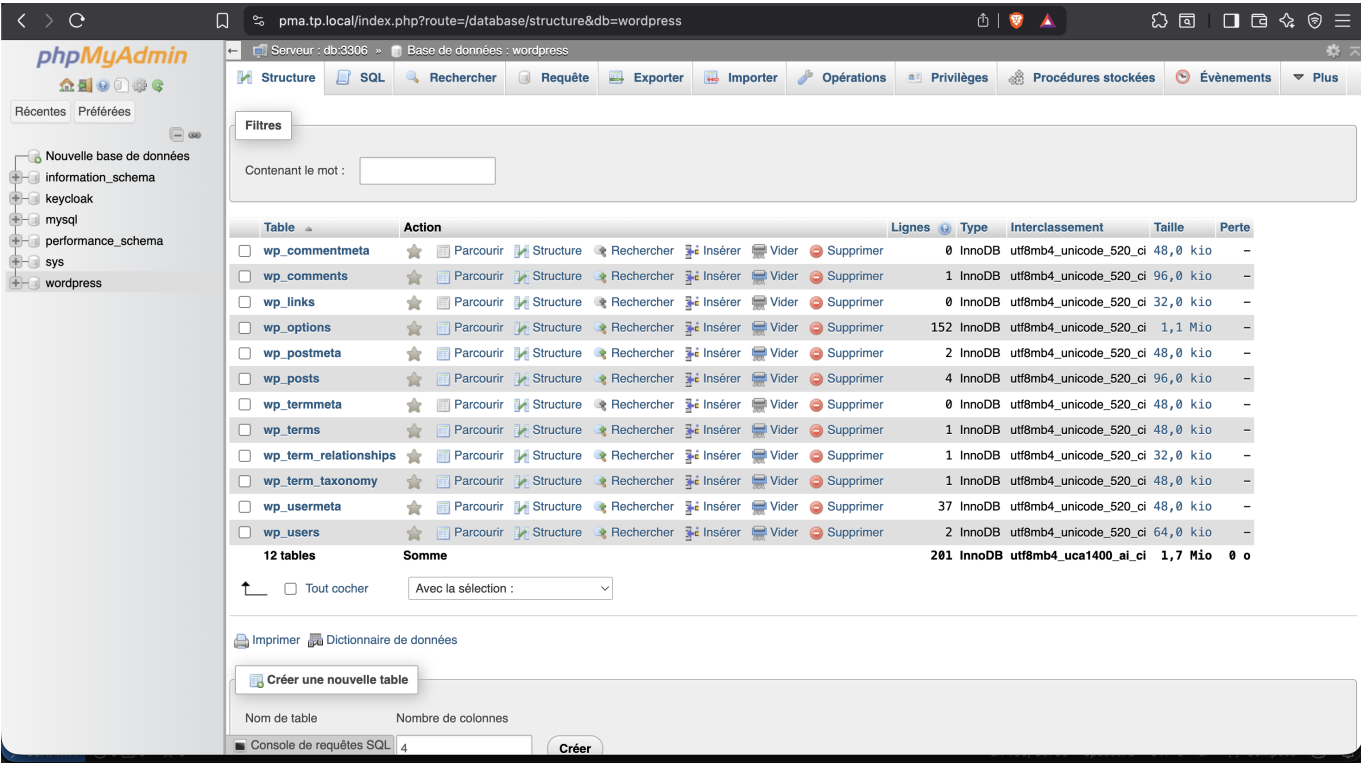


Page de profil WordPress du user "testuser" (Test User), provisionné automatiquement par le plugin OpenID Connect Generic après authentification via Keycloak. Prouve que le SSO fonctionne de bout en bout.

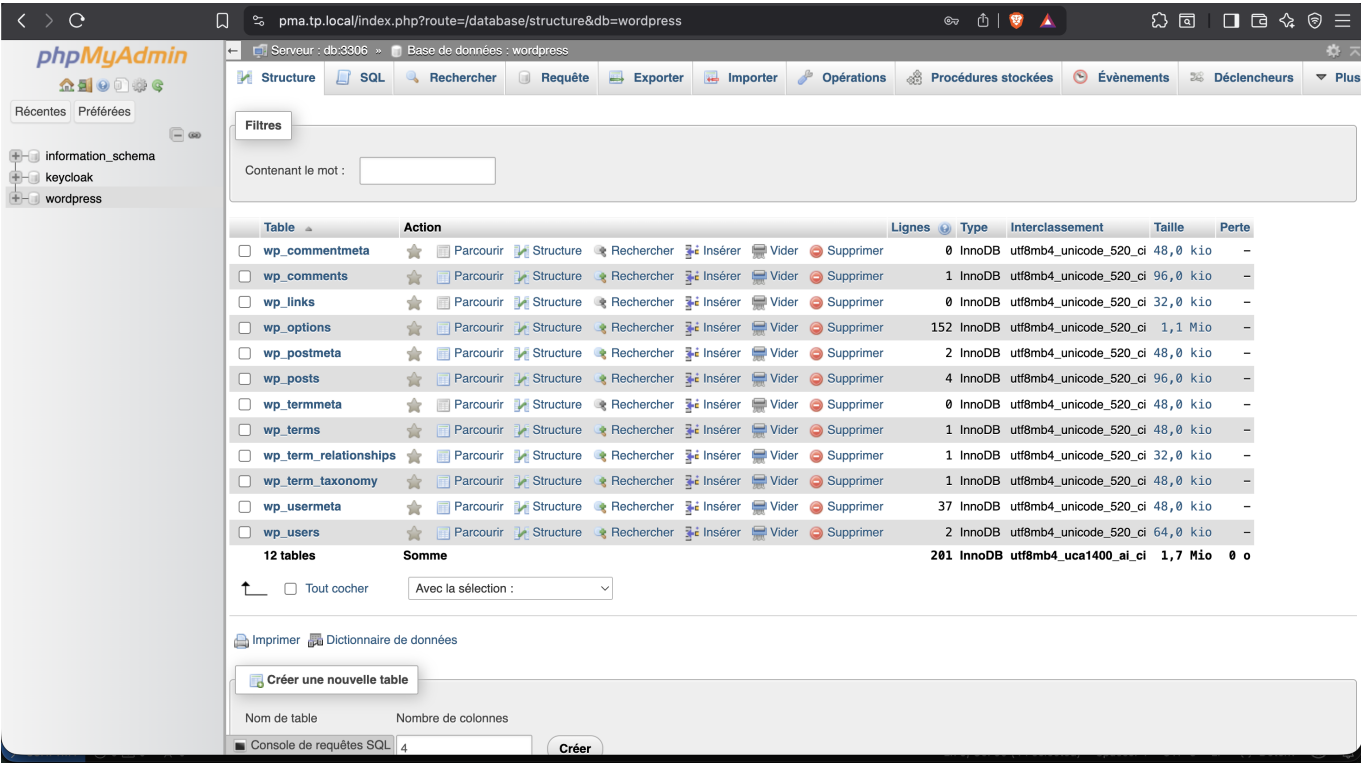
VM-App : phpMyAdmin



Première couche de sécurité : popup d'authentification HTTP Basic Auth (middleware Traefik `pma-auth`) avant d'accéder à phpMyAdmin sur `pma.tp.local`.

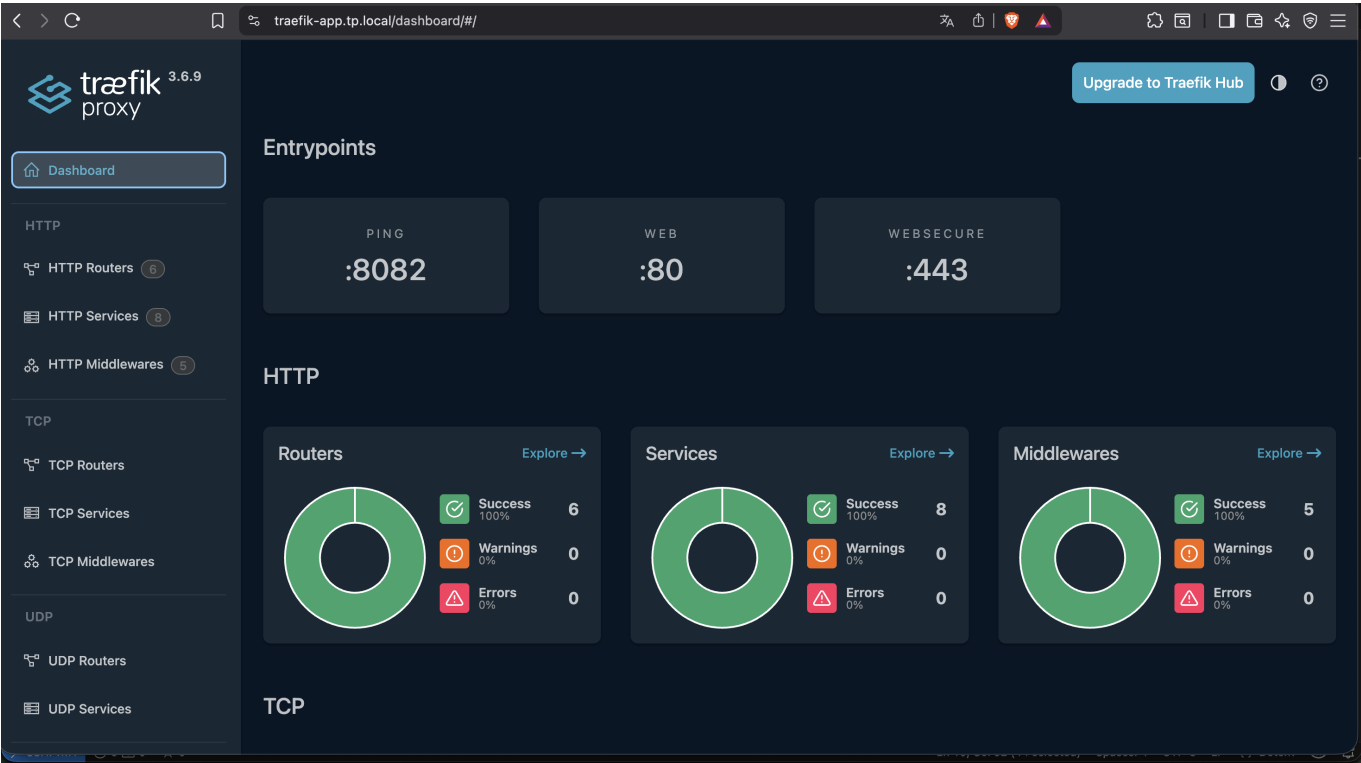


Interface phpMyAdmin connectée en tant que **root**. Toutes les bases sont visibles (information_schema, keycloak, mysql, performance_schema, sys, wordpress). La base **wordpress** affiche ses 12 tables.



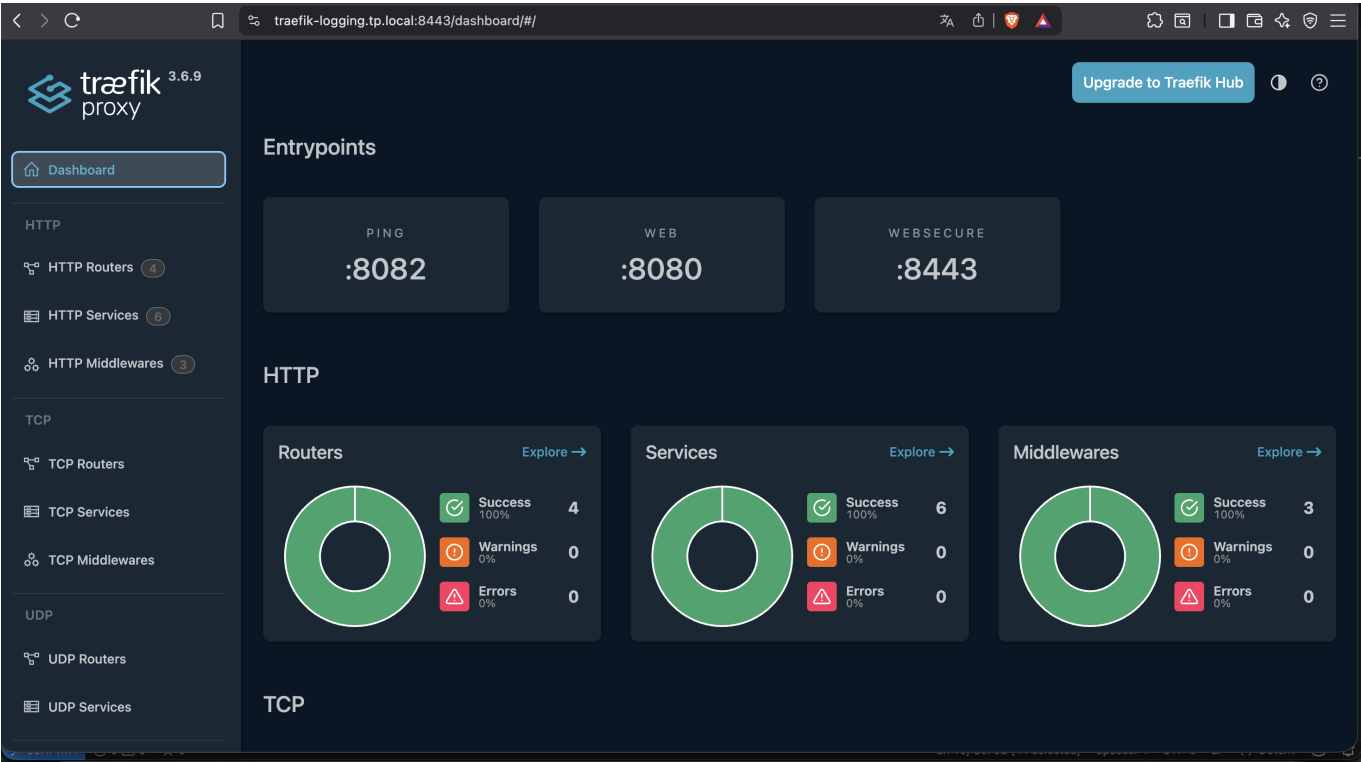
Interface phpMyAdmin connectée en tant que **wpdbadmin**. Seules les bases autorisées sont visibles (information_schema, keycloak, wordpress), démontrant la restriction des privilèges au niveau MariaDB.

VM-App : Traefik Dashboard



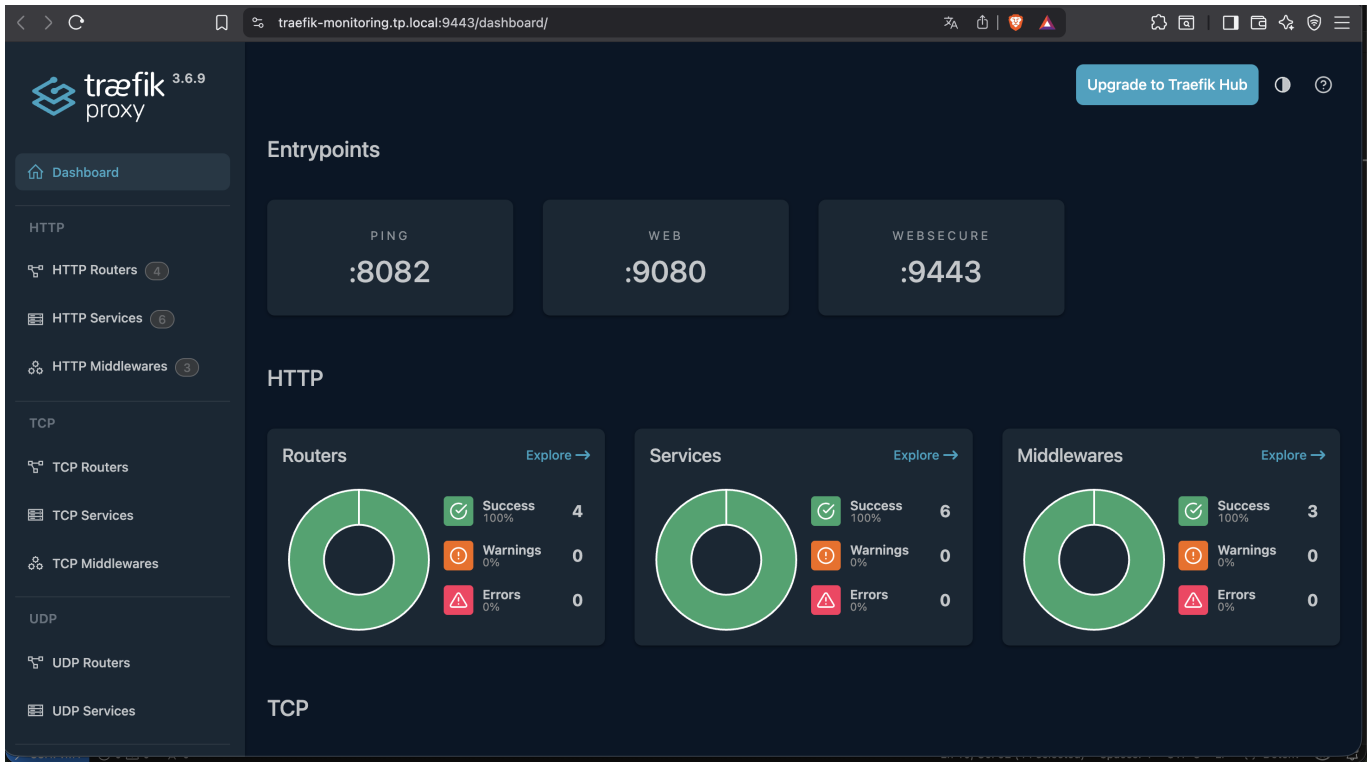
Dashboard Traefik v3.6.9 de la VM App (traefik-app.tp.local). 3 entrypoints (PING :8082, WEB :80, WEBSECURE :443), 6 routers HTTP, 8 services et 5 middlewares — tous à 100% sans erreurs.

VM-Logging : Traefik Dashboard



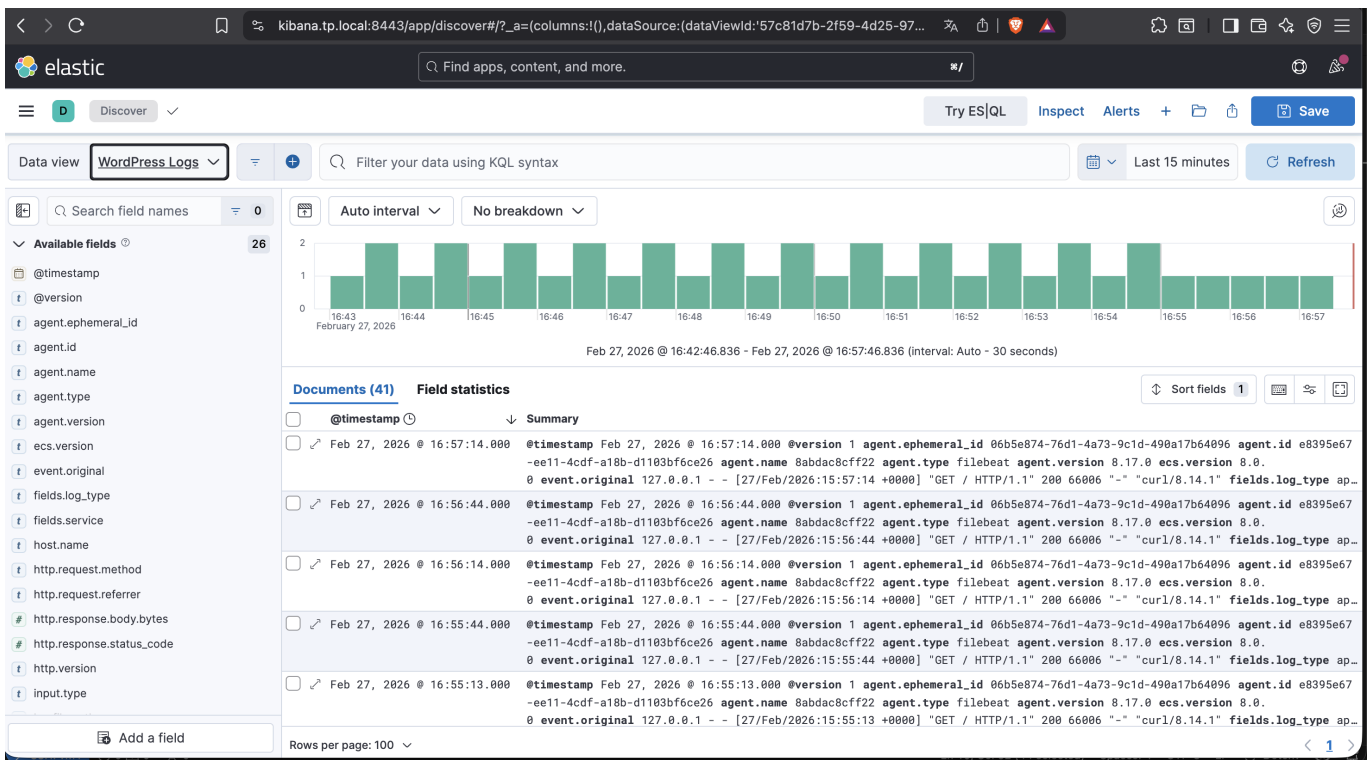
Dashboard Traefik de la VM Logging (traefik-logging.tp.local:8443). Entrypoints sur ports non-standard (8080/8443). 4 routers, 6 services, 3 middlewares, tous healthy.

VM-Monitoring : Traefik Dashboard

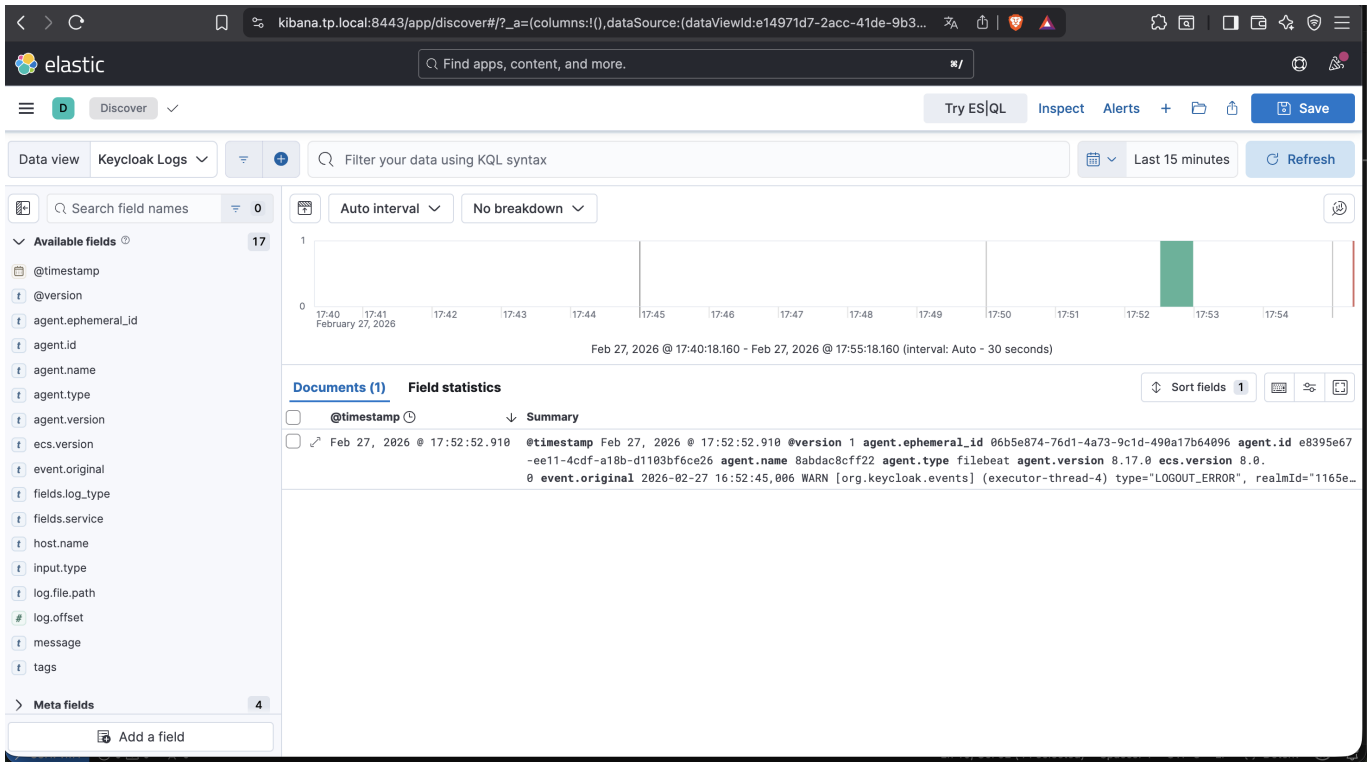


Dashboard Traefik de la VM Monitoring (`traefik-monitoring.tp.local:9443`). Entrypoints sur 9080/9443. 4 routers, 6 services, 3 middlewares, tous healthy.

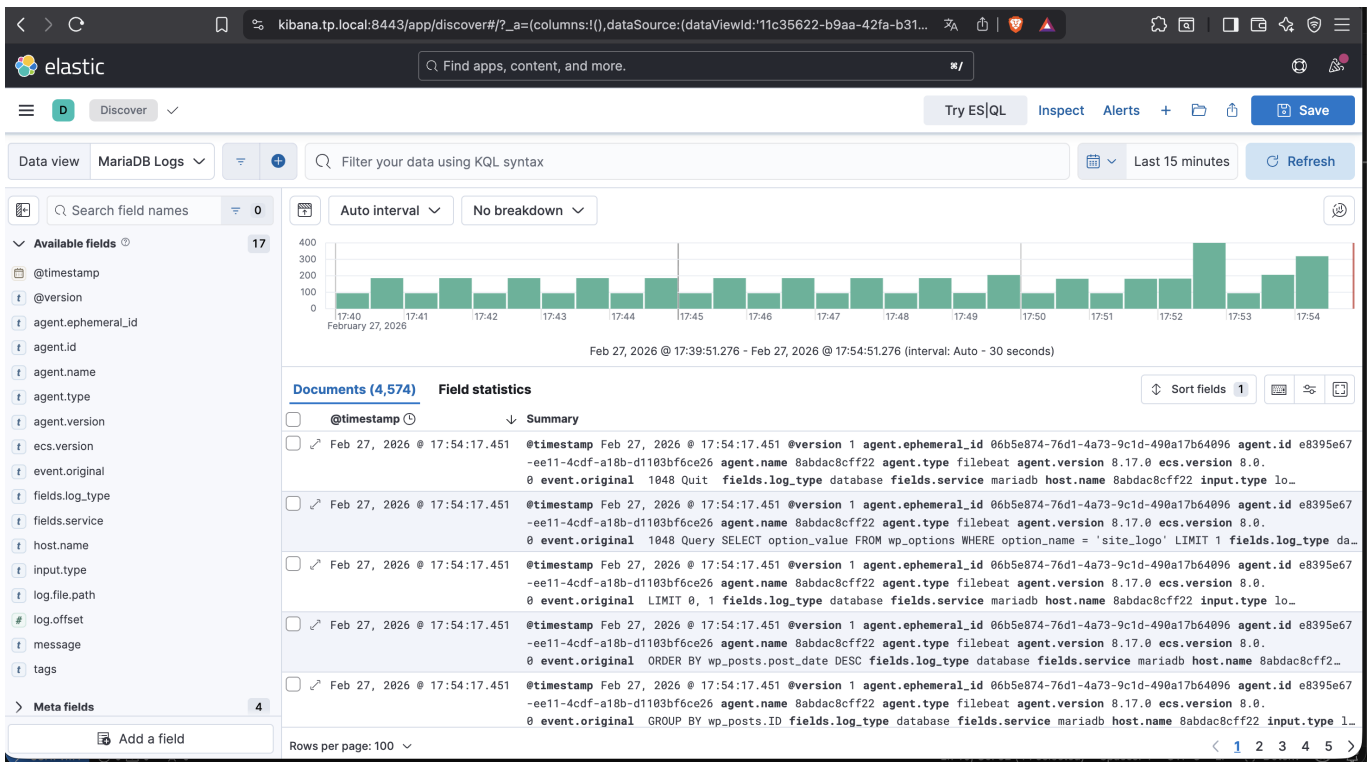
VM-Logging : Kibana — Logs par service



Vue Kibana Discover filtrée sur les logs WordPress. Histogramme temporel des événements + détail des entrées Apache access log (requêtes GET, codes HTTP, user-agents). Confirme l'ingestion des logs WordPress dans Elasticsearch.

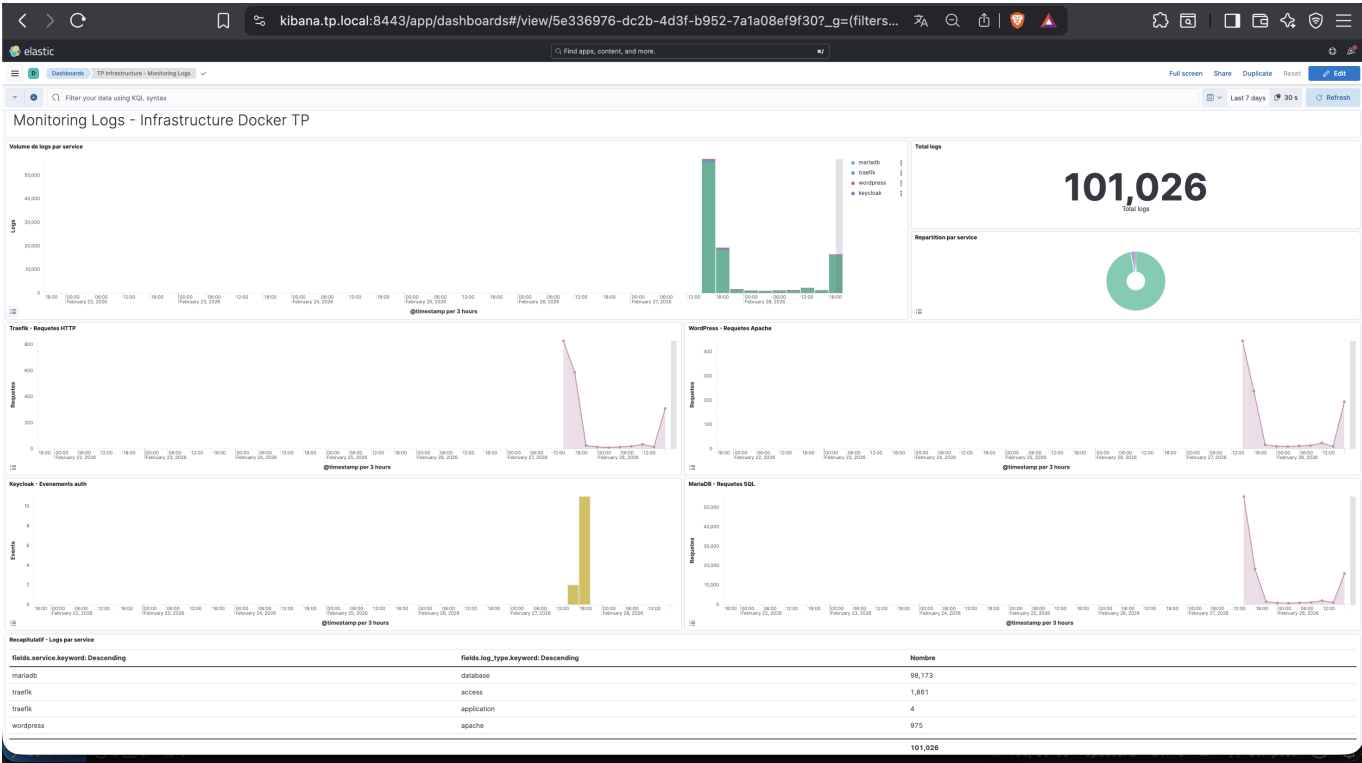


Vue Kibana Discover filtrée sur les logs Keycloak. Entrée visible de niveau WARN (`org.keycloak.events`, type `LOGOUT_ERROR`). Confirme la remontée des logs applicatifs Keycloak.



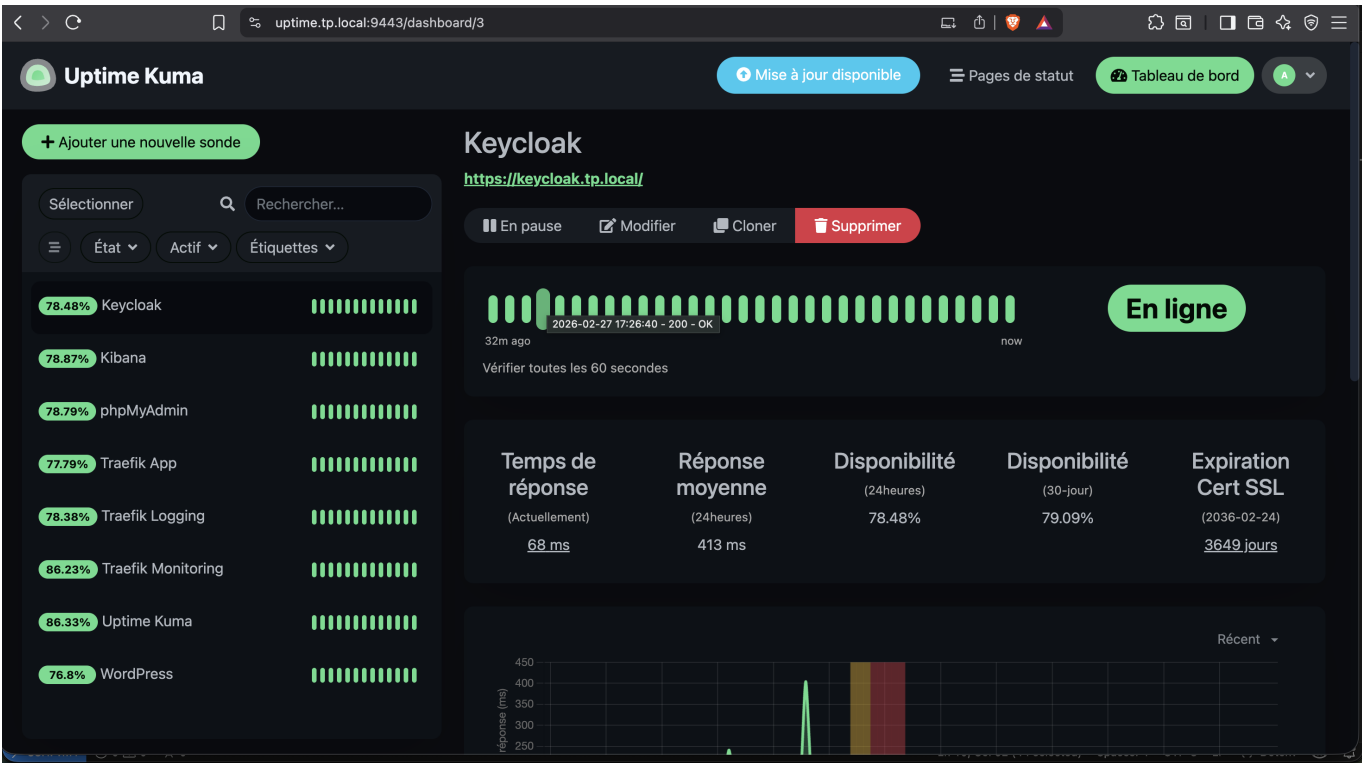
Vue Kibana Discover filtrée sur les logs MariaDB — 4 574 documents indexés. Entrées SQL visibles (SELECT, GRANT, GROUP BY). Confirme l'ingestion des query logs MariaDB.

VM-Logging : Kibana — Dashboard de synthèse



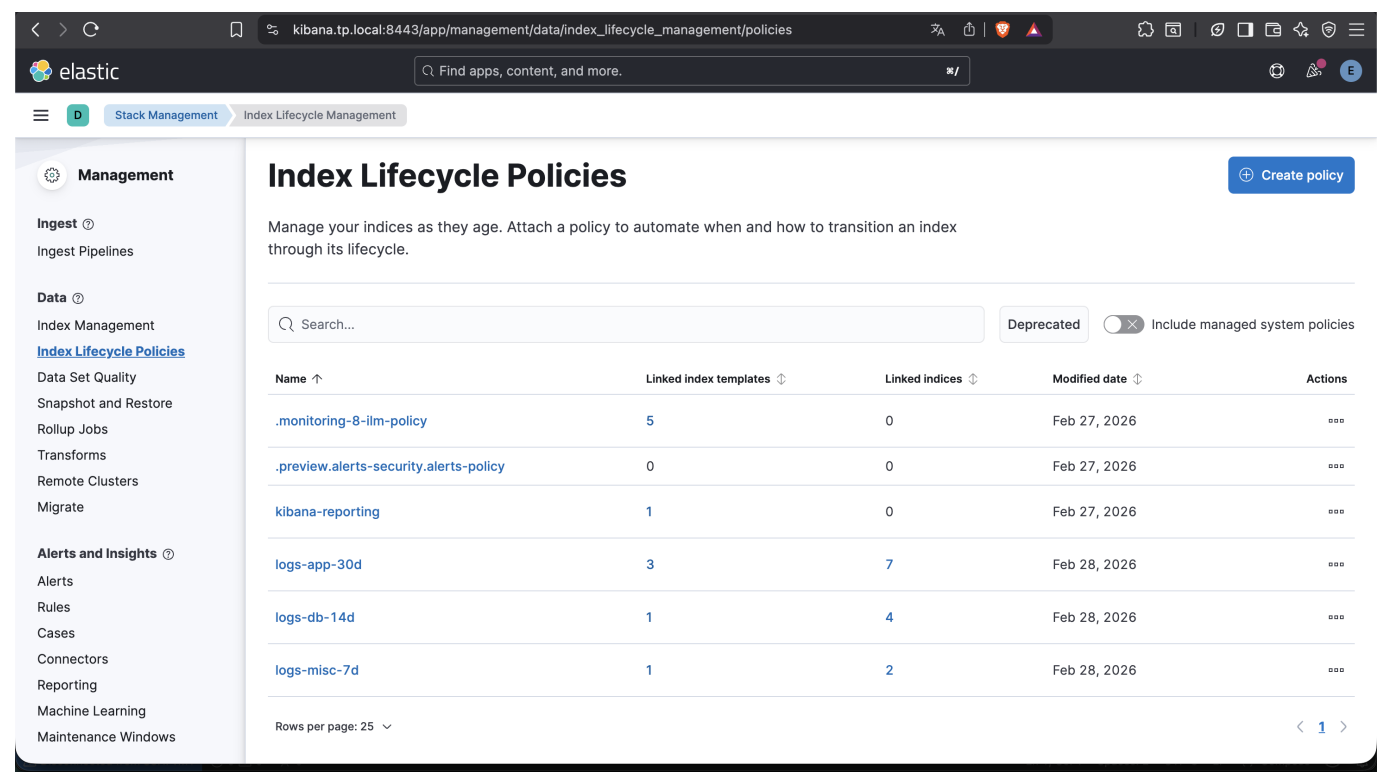
Dashboard custom "Monitoring Logs - Infrastructure Docker TP" avec vue d'ensemble : **101 026 logs** au total, histogramme par service, répartition en pie chart, et séries temporelles par type (Traefik HTTP, WordPress Apache, Keycloak events, MariaDB SQL). Table de synthèse par service et type de log en bas.

VM-Monitoring : Uptime Kuma

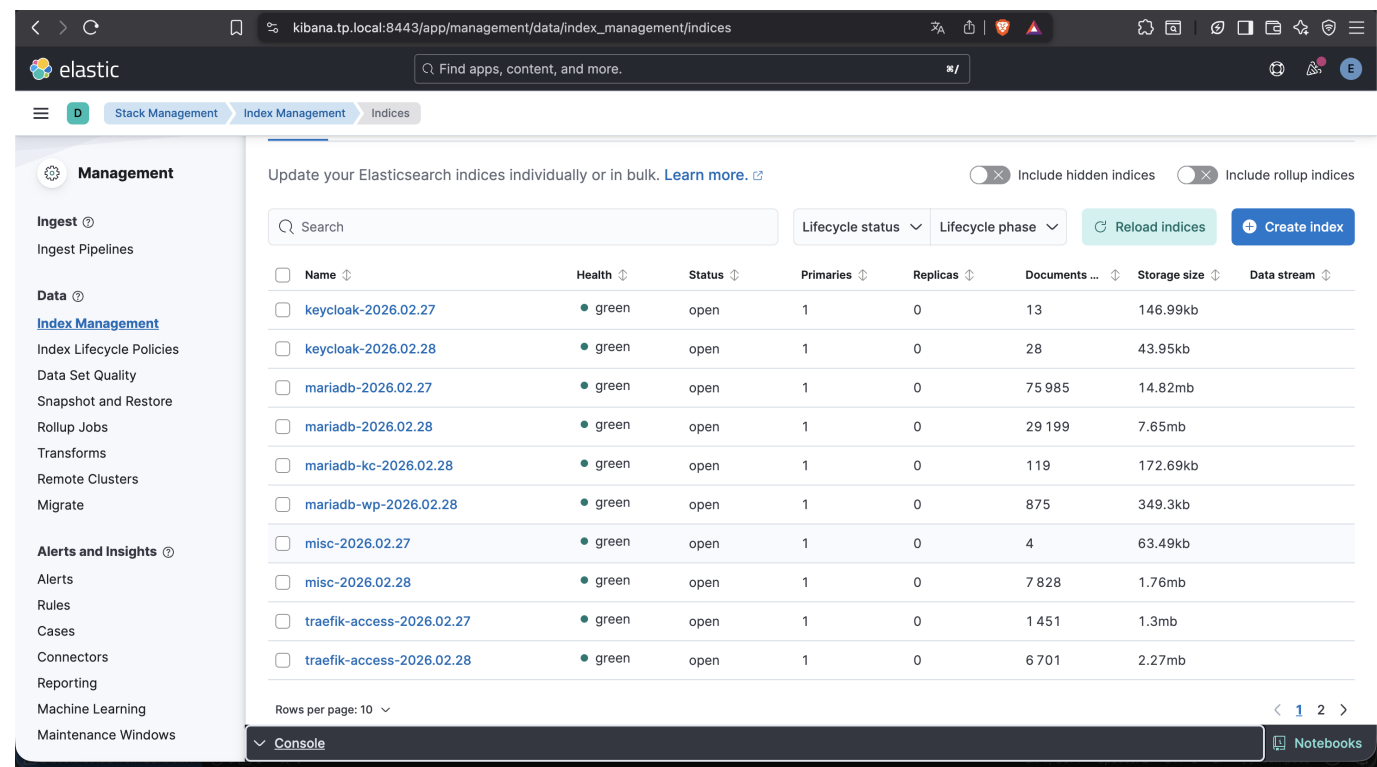


Dashboard Uptime Kuma sur uptime.tp.local:9443 surveillant 8 services : Keycloak, Kibana, phpMyAdmin, Traefik App/Logging/Monitoring, Uptime Kuma et WordPress. Tous les services sont en ligne. Détail Keycloak sélectionné : "En ligne", 68ms de réponse.

VM-Logging : Kibana — ILM Policies

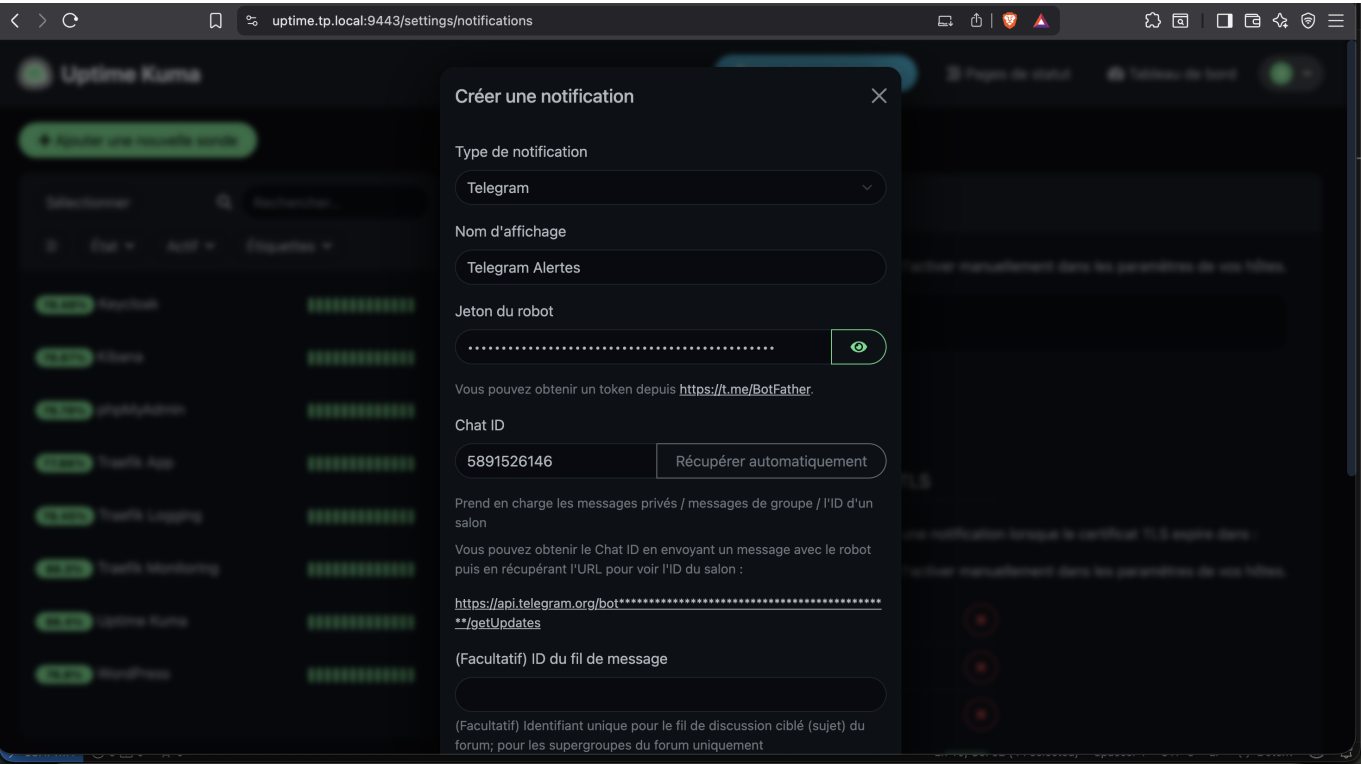


Page Stack Management → Index Lifecycle Policies. Les 3 policies custom sont visibles : **logs-app-30d** (3 templates, 7 indices), **logs-db-14d** (1 template, 4 indices), **logs-misc-7d** (1 template, 2 indices). Chaque policy gère automatiquement la suppression des vieux logs.

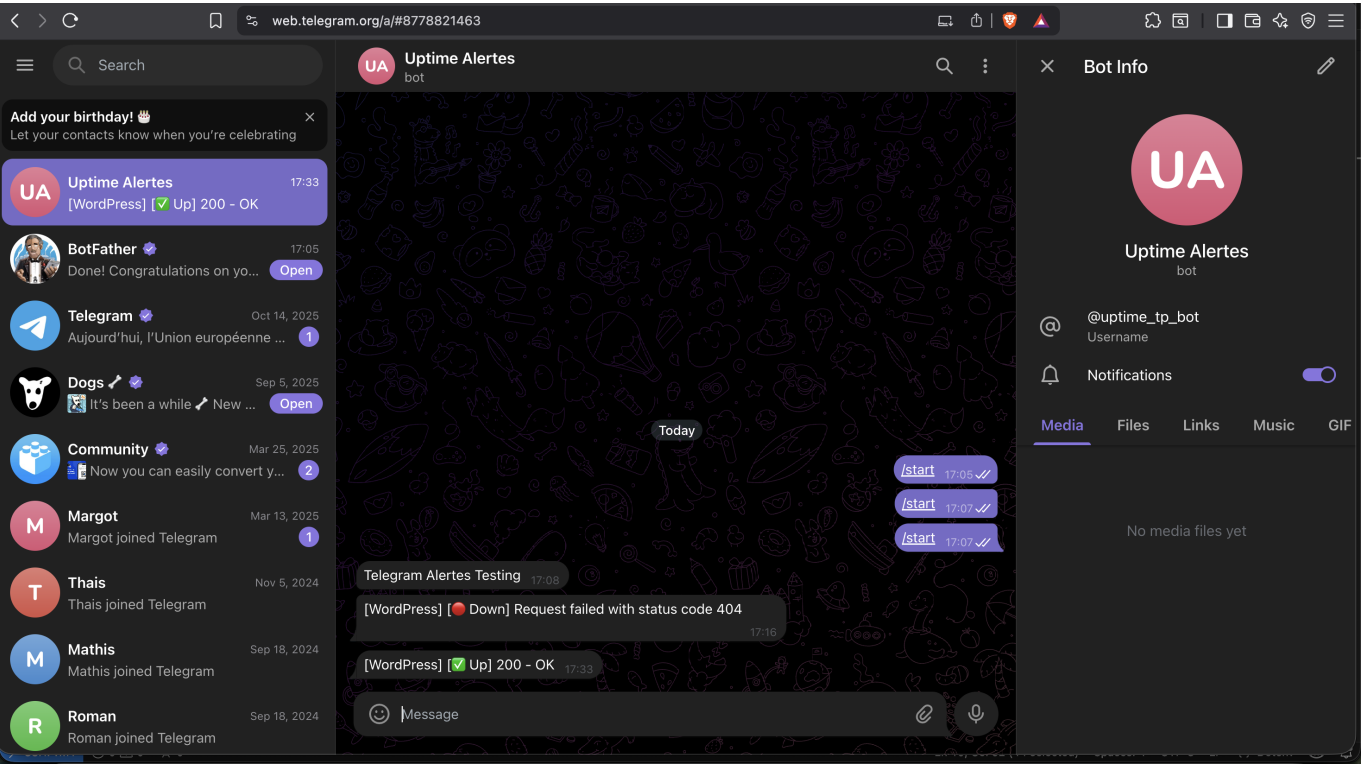


Page Stack Management → Index Management. Tous les indices applicatifs sont listés avec leur statut **green**, 0 replicas, et la policy ILM rattachée. On voit les nouveaux indices **mariadb-wp-*** et **mariadb-kc-*** séparés suite au split des stacks.

VM-Monitoring : Alerting Telegram



Configuration de la notification Telegram dans Uptime Kuma : bot token, Chat ID et URL de l'API Telegram Bot. Canal "Telegram Alertes" lié à tous les monitors.



Conversation Telegram avec le bot "Uptime Alertes" montrant les notifications en temps réel : alerte DOWN ([WordPress] [Down] Request failed with status code 404) suivie d'une alerte UP ([WordPress] [Up] 200 - OK). Prouve le fonctionnement de l'alerting de bout en bout.